



# Chapter 10: Classes and Data Abstraction PART I

# Objectives

In this chapter, you will:

- Learn about classes
- Learn about `private`, `protected`, and `public` members of a class
- Explore how classes are implemented
- Become aware of accessor and mutator functions
- Examine constructors and destructors

# Objectives (cont'd.)

- Learn about the abstract data type (ADT)
- Explore how classes are used to implement ADTs
- Become aware of the differences between a `struct` and a `class`
- Learn about information hiding
- Explore how information hiding is implemented in C++
- Learn about the `static` members of a class

# Objects in Real Environment

- Everything surrounded us is an **object**.
- An object is **animate** or **inanimate**
  - Animate objects: human, animal.
  - Inanimate objects: car, football, door, etc.
- Objects either animate and inanimate have:
  - **Behavior** (Human: think, eat, drink, sleep, etc.)
  - **Attributes** (Human: name, age, weight, etc.)

# Objects in Real Environment (cont'd.)

- Objects communicate with each other via **sending message**
  - Example: a sergeant commanding a soldier to stand at attention
- The communication between objects is performed using **object interfaces**
  - Examples: (Door: handle, Car: steering wheel, Human: mouth/ears)
- There are different relationships between objects
  - **Inheritance**
  - **Composition**

# Objects in Real Environment (cont'd.)

– Objects hide their internal details from surrounding environment for two reasons:

**1. To protect object contents from tampering**

- Example1: human skin protect the body human against versus, sunlight, air, radiation, etc.

**2. To easily use an object without paying attention to object details (internal implementation)**

- Example2: you can drive a car effectively without knowing the details of how the engine system work internally

# Object-Oriented Programming (OPP)

- OOP models real-objects with software counterparts
  - OOP models real-world objects' attributes and behavior.
  - OOP models communication between real-world objects.
  - OOP models relationships between real-world objects.
- OOP encapsulates data (attributes) and methods (behavior) into a package called **object**
- This encapsulation is called **information hiding**
  - Information hiding means hiding the internal implementation of an object against the real environment.

# Classes and Objects

- To model real-world objects we should use **classes** and **objects**
  - A **class** is a blueprint for objects
  - An **object** is an instantiation (example) of a class
  - The relationships between class and object is the same relationship between **house blueprint** and the **physical house**
- A class consists of a set of components each one called **member**



# Classes and Objects (cont'd.)

- Class definition:
  - Defines a data type; no memory is allocated
  - Don't forget the semicolon after the closing brace
- Syntax:

```
class classIdentifier  
{  
    classMembersList  
};
```

# Classes (cont'd.)

- Class member can be a variable or a function
- If a member of a `class` is a variable
  - It is declared like any other variable
  - You cannot initialize a variable when you declare it
- If a member of a `class` is a function
  - Function prototype is listed
  - Function members can (directly) access any member of the `class`

# Classes (cont'd.)

- Three categories of class members:
  - `private` (default)
    - Member cannot be accessed outside the `class`
  - `public`
    - Member is accessible outside the class
  - `protected`

# Variable (Object) Declaration

- Once defined, you can declare variables of that `class` type

```
Point p1;
```

- A `class` variable is called a class object or class instance

FIGURE 10-2 Objects `myClock` and `yourClock`

# Accessing Class Members

- Once an object is declared, it can access the `public` members of the class
- Syntax:

`classObjectName.memberName`

  - The dot ( `.` ) is the **member access operator**
- If an object is declared in the definition of a member function of the `class`, it can access the `public` and `private` members

# Built-in Operations on Classes

- Most of C++'s built-in operations do not apply to classes
  - Arithmetic operators cannot be used on class objects unless the operators are overloaded
  - Cannot use relational operators to compare two class objects for equality
- Built-in operations that are valid for class objects:
  - Member access (.)
  - Assignment (=)

# Implementation of Member Functions

- Must write the code for functions defined as function prototypes
- Prototypes are left in the class to keep the class smaller and to hide the implementation
- To access identifiers local to the class, use the scope resolution operator ::

# Implementation of Member Functions (cont'd.)

- Once a `class` is properly defined and implemented, it can be used in a program
  - A program that uses/manipulates objects of a class is called a client of that class
- When you declare objects of the `class Point`, each object has its own copy of the member variables (`x` and `y`)
  - Called instance variables of the class
    - Every object has its own instance of the data



# Accessor and Mutator Functions

- Accessor function: member function that only accesses the value(s) of member variable(s)
- Mutator function: member function that modifies the value(s) of member variable(s)

# Order of `public` and `private` Members of a Class

- C++ has no fixed order in which to declare `public` and `private` members
- By default, all members of a class are `private`
- Use the member access specifier `public` to make a member available for `public` access

# Constructors

- Use constructors to guarantee that member variables of a class are initialized
- Two types of constructors:
  - With parameters
  - Without parameters (default constructor)
- Name of a constructor = name of the class
- A constructor has no type

# Constructors (cont'd.)

- A class can have more than one constructor
  - Each must have a different formal parameter list
- Constructors execute automatically when a class object enters its scope
- They cannot be called like other functions
- Which constructor executes depends on the types of values passed to the class object when the class object is declared

# Invoking a Constructor

- A constructor is automatically executed when a class variable is declared
- Because a class may have more than one constructor, you can invoke a specific constructor

# Invoking the Default Constructor

- To invoke the default constructor:

```
className classObjectName;
```

- Example:

```
clockType yourClock;
```

# Invoking a Constructor with Parameters

- Syntax:

```
className classObjectName(argument1, argument2, ...);
```

- Number and type of arguments should match the formal parameters (in the order given) of one of the constructors
  - Otherwise, C++ uses type conversion and looks for the best match
  - Any ambiguity causes a compile-time error

# Constructors and Default Parameters

- A constructor can have default parameters
  - Rules for declaring formal parameters are the same as for declaring default formal parameters in a function
  - Actual parameters are passed according to same rules for functions
- Default constructor: a constructor with no parameters or with all default parameters



# Classes and Constructors: A Precaution

- If a class has no constructor(s), C++ provides the default constructor
  - However, object declared is still uninitialized
- If a class includes constructor(s) with parameter(s), but not the default constructor
  - C++ does not provide the default constructor

# Destructors

- Destructors are functions without any type
- The name of a destructor is the character '~' followed by class name
  - For example:  
`~clockType();`
- A class can have only one destructor
  - The destructor has no parameters
- Destructor automatically executes when the class object goes out of scope

# Const Objects and Const Member Function

- Constant Object
  - Object is not modifiable
- Constant function:
  - Member function that cannot modify member variables
  - Use `const` in function heading

# Pointer of Object

```
int main()
{
    Point*p1; // declare pointer (p1) of object
    p1 = new Point;
    p1->setXY(10,20); //sending message
    p1->print(); //sending massgae
}
```

# The End